

TianShanOS

Complete Guide to the Security Page

Set up, check, and maintain your device's security settings

[Account security](#) · [SSH keys](#) · [Host fingerprints](#) · [HTTPS / mTLS](#) · [Config Pack](#)

Generated: 2026-09-07

Contents

1. Before you begin	3
2. Change device passwords	4
3. Connect to a server with an SSH key	5
4. Manage keys, hosts, and fingerprints	7
5. Configure HTTPS certificates and mTLS	9
6. Understand Config Pack limitations	11
7. Troubleshooting and incident response	13
8. Review scope and glossary	14

Use this guide to change device passwords, manage SSH access, and configure certificates. If you are setting up the device for the first time, start with Chapters 1 and 2. Certificate and Config Pack procedures are intended for administrators responsible for device security.

Version checked: the working copy reviewed on September 7, 2026, based on commit [d6ed947](#). This guide reflects a source-code review, not testing on your device. Carry out the checks in each section in your own environment.

Use an isolated management network. The full web interface currently uses HTTP, and most API operations lack enforced, centralized login and permission checks. Do not expose the device to the internet or a guest network. Installing an HTTPS certificate does not switch the full web interface to HTTPS.

1. Before you begin

1.1 Find the task you need

Task	Where to go
Change the device's root or admin password	Account Security; Chapter 2
Connect to a server using an SSH key	Key Management and Deployed Hosts; Chapter 3
Copy keys or check a server's identity	Key Management and host fingerprints; Chapter 4
Set up certificates and mutual authentication	HTTPS Certificate; Chapter 5
Exchange encrypted configuration packages	Config Pack; Chapter 6. General configuration application is not implemented yet

1.2 Accounts and device identity

- **admin** can open the Security page and see keys, hosts, certificates, and Config Pack controls, but not Account Security.
- **root** can also set the root and admin passwords or reset admin to its default password. These password-management operations enforce root authorization on the server.
- A **Developer device** is identified through the OU field in its device certificate. This is not a user account. Only these devices can currently export Config Packs and SSH host configurations; signing in as root does not change the device's identity.

Except for operations such as password management that check authorization themselves, a login screen does not prevent direct API access. Network isolation remains a requirement.

1.3 Prepare for the operation

1. Check that the IP address in your browser belongs to the intended device and that your computer is on a trusted management network.
2. Confirm the SSH server address, port, and username with its administrator.
3. To deploy or revoke a public key through this page, you need the remote account's password, and that account must accept password authentication. Do not leave server restrictions relaxed just to troubleshoot.
4. Before deleting keys, revoking access, or replacing certificates, make sure you have another way in: a server console, a separate administrator key, or the device's HTTP management interface.

2. Change device passwords

These controls change **TianShanOS login passwords**, not the SSH password on a remote server.

2.1 The password prompt after sign-in

If an account is still marked as having an unchanged password, a password-change prompt appears after sign-in. Set a long, unique password. Choosing to change it later dismisses the prompt; it does not make the default password safe to keep.

This prompt is not an account-settings screen that you can reopen at any time. If admin has already changed its password and needs another change, root can set it from the Security page.

2.2 Set a password as root

1. Sign in as root and open Security.
2. Under Account Security, select **Set root password** or **Set admin password**.
3. Enter the same new password twice and submit. The interface accepts 4-64 characters; four characters is a technical minimum, not a security recommendation.

Check the result: sign in with the new password in a private browsing window before closing your original session. Changing a password does not automatically end existing sessions.

If sign-in fails: check the account and device address first. Five consecutive failed attempts trigger a lockout of about five minutes. Avoid repeated guesses.

2.3 Reset the admin password

Root can reset admin to the default password, `rm01`, and clear its login lockout. After resetting, sign in as admin in a new session and set a new password immediately.

A reset is a temporary recovery measure. Do not keep the default password, and do not assume a password change signs out every existing session.

3. Connect to a server with an SSH key

For initial setup: **create an RSA key, deploy its public key, test the connection, then check the server fingerprint.** When retiring a key, **revoke access on the servers first, confirm that the old key no longer works, and only then delete the local key.**

3.1 Create a key

1. In Key Management, select **Generate New Key**.
2. Check the existing list and choose an unused key ID, such as `backup01`. Do not reuse an ID: the generation API does not reject duplicates and may overwrite the existing key.
3. Choose RSA 2048 or RSA 4096. RSA 2048 is the interface default. ECDSA options are shown, but the current SSH public-key authentication path does not support them.
4. Add a comment or alias if useful. Allow private-key export only if you need it for backup or migration; there is no interface for changing this setting later.
5. Select Generate, wait for completion, and refresh the list.

Check the result: find the intended ID and RSA type. Open Public Key and check that the complete text starts with `ssh-rsa`.

Keep in mind: the key list holds eight entries. Do not keep generating keys when it is full; retire an unused key first. Key storage and list registration can succeed or fail separately, so check the list even after a success message. Hide Key ID does not prevent the API from exposing the real ID.

3.2 Deploy the public key

Deployment appends the public key to the remote account's `~/.ssh/authorized_keys`, allowing that account to accept the matching private key. It does not change the server password.

1. Select Deploy on the new key's row.
2. Enter the server address, username, SSH port, and that account's password.
3. Review the details and select Start Deploy.
4. Check for the server under Deployed Hosts, then test it as described below.

The current implementation authenticates with the password before checking the host fingerprint, and automatically trusts previously unknown hosts. It does not guarantee that the server's identity is verified before the password is sent. Make the first connection on a controlled network; checking the fingerprint afterward cannot undo password exposure.

If deployment reports success but the record is missing or the test fails: do not immediately deploy again. The public key may already be installed even though the follow-up key test or local record save failed. Check `authorized_keys` through another management connection. Repeated deployment can append duplicate entries.

3.3 Test the connection

1. Under Deployed Hosts, check the address, port, username, and key ID.
2. Select Test. The device attempts to run `echo "TianshanOS SSH Test OK"`.
3. After the first connection, check the saved host fingerprint using Section 4.3.

What success means: the page checks whether the API operation succeeded, but does not check the remote command's exit status or output. Treat this as a basic connection check, not a complete acceptance test. It does not establish that `sudo` or your application commands will work. For an important workload, verify the actual command's output, exit status, and permissions.

If the test fails: check connectivity, the SSH service, the account, and remote authorization. Stop retrying if the fingerprint has changed or the server's identity is uncertain; follow Section 4.3.

3.4 Revoke access before deleting the local key

1. Confirm that another server-management connection works and that the original key still exists on the device.
2. Select Revoke on the host row, enter the server password, and confirm **Revoke & Remove**. You can also start from the key row and enter the target details.
3. Review the result, then use a trusted server-management connection to check that the public key and any duplicate entries are gone from `authorized_keys`.
4. Confirm that the old key can no longer sign in. Revocation leaves an `authorized_keys.bak` backup; handle it under the server's backup policy so an old authorization is not restored later.
5. Refresh the local host list. The page attempts to remove the record but does not check that request's result. Remove the record manually if it remains.
6. Only after every target server is handled should you delete the old key from the device and remove unneeded private-key copies.

If no matching public key is found: confirm on the server that you selected the correct account and that the key is absent before choosing to remove only the local record.

If revocation fails or the server looks suspicious: keep the local key and use a trusted console or another administrator connection. Do not keep sending a password to a server whose identity is in doubt.

4. Manage keys, hosts, and fingerprints

4.1 Copy a public key or export a private key

Public key: select Public Key and copy the complete, single-line value for the server administrator. A public key is not secret. Sending it to the wrong person does not by itself expose the private key or require key replacement. Access is granted only when an administrator adds it to an account's authorization list.

Private key: export is available only if you enabled it when the key was created. Use a trusted computer on an isolated network, store the key in an approved secret store, and clear temporary copies from the clipboard and download folder. Never paste it into chat, tickets, or logs.

If Copy does nothing: browsers may block clipboard access on an HTTP page. Select and copy the visible text manually, then check that the beginning, end, and full contents are intact. Do not weaken browser security settings just to enable copying.

4.2 Revoke, Remove, and Delete are different

Action	Effect
Revoke a public key	Attempts to remove authorization on the server; requires the remote password
Remove a host	Deletes the device's local connection record, not remote authorization
Delete a key	Deletes local key material; does not contact the server or revoke access
Delete a host fingerprint	Deletes a saved server-identity record, not the connection record or remote authorization

Deployed Hosts is a local list, not a live view of server authorization. An empty list does not prove that remote access has been removed, and a listed host is not guaranteed to be reachable.

4.3 Check the SSH server fingerprint

A fingerprint identifies the server you are connecting to. This page stores the SHA-256 digest as **64 hexadecimal characters**. OpenSSH tools commonly show `SHA256:base64` instead. Ask the administrator to provide the same format before comparing; the strings are not directly interchangeable.

After the first connection: open the full fingerprint with View in the host-fingerprint section. Compare it with a value obtained from a server console, asset inventory, or another trusted channel. The table shows only the first 32 characters, which is not enough for the full check.

If they differ: stop connecting and investigate in isolation. If you entered an SSH password, treat it as potentially exposed. Use a trusted connection to change it, review sign-in logs, and remove any unwanted public-key authorization. Do not use the suspicious connection to revoke access through this page.

If a previously saved fingerprint changes:

1. Stop retrying. Do not simply delete the old entry.
2. Use a trusted console to verify the new fingerprint, IP address, port, asset identity, and maintenance record.
3. Delete the old fingerprint only after confirming an authorized server rebuild or host-key change.
4. Reconnect on a controlled network, then view and check the new saved fingerprint.

The current page may show a generic connection error rather than a dedicated fingerprint-comparison dialog.

Protect the SD card: fingerprints are stored locally and synchronized to plaintext JSON on the card. At startup, available SD fingerprint configurations replace the corresponding NVS records. These files are not signed; do not allow untrusted changes to them.

4.4 Import and export SSH host configurations

This dedicated `.tscfg` workflow is separate from the unfinished general configuration application in Chapter 6. A host package contains the address, port, username, authentication type, and key ID. **It contains neither the SSH password nor the private key, and it does not grant access on the server.**

To export: on a Developer device, select Export on the host row. Supply and verify the target device's certificate when exporting for another device, then download the package. An ordinary device may show the button, but the backend rejects its export request.

To import:

1. Confirm that the package was made for this device, the SD card is writable, and the source has been verified through a trusted channel.
2. Make sure this device already has the correct key referenced by the package. A matching ID alone is not enough; the key material must match the server's authorization.
3. Select Import Host, choose the file, and review the preview. Enable overwrite only when you intend to replace an existing configuration with the same name.
4. Confirm and restart as prompted. Import saves the package to the SD card; loading and decryption are attempted at restart.
5. Check the loaded address, username, port, and key ID, then test the connection.

A successful preview is not proof that the package will work. It neither establishes trust in the signer nor checks the recipient fingerprint. The target check happens during loading after restart. Remove an incorrect record and its corresponding SD package to prevent it from loading again. Before replacing the device certificate, also read Section 5.5.

5. Configure HTTPS certificates and mTLS

This chapter is for certificate administrators. The current service on port 443 offers only health, identity, and permission-test endpoints, **not the full web interface**. With its default settings, startup requires a device private key, a device certificate, and a client CA chain.

5.1 Know which certificate does what

- The **device certificate and private key** let the device prove its identity to a connecting client.
- A **client certificate and private key**, held by a computer or service, let that client prove its identity to the device.
- The **CA chain installed on the device** is used to verify client certificates. It does not automatically make a computer or browser trust the device certificate.

This two-way certificate exchange is called mutual TLS, or mTLS. The client must still trust the CA that issued the device certificate and check its name and permitted uses.

5.2 Generate a device key and certificate request

1. Under HTTPS Certificate, select Generate Key Pair. This creates a separate ECDSA P-256 private key, unrelated to SSH keys. It cannot be exported through this interface.
2. If a key already exists, stop and review Section 5.5 before proceeding. Generating another key overwrites the old one.
3. Select Generate CSR. Enter the device ID (CN), organization (O), and organizational unit (OU), or leave all fields blank.
4. Send the complete CSR text to your CA administrator. A CSR requests a certificate; it contains no private key and does not install a certificate.

Check names before issuance: the custom-field path does not generate a SAN. With all fields blank, the CN is fixed at `TIANSHAN-DEVICE-001`; the current IP is included as an IP SAN only if it can be obtained. No DNS SAN is added. Have your CA administrator inspect the CSR and use a controlled issuance process to put the required IP addresses or DNS names in the final certificate's SAN. This form cannot edit SANs.

Check the result: refresh the page to confirm that the private key exists. Have the CA administrator inspect the CSR's public key, subject, and SAN, and ensure that the issued certificate has the required server-authentication purpose (EKU).

5.3 Install the device certificate

1. Obtain a PEM certificate that matches the device's current private key.
2. Select Install Cert, paste the full text including its boundary markers, and submit.
3. View the certificate and check its subject, issuer, and validity dates.

The success message means only that the certificate can be parsed and its public key matches the current private key. Installation does not fully validate the trust chain, SAN, EKU, current validity, or subject policy.

The actual client must check the chain, access name, purposes, and validity. If installation reports a key mismatch, locate the certificate issued for the current CSR. Do not generate a new private key simply to clear the error.

5.4 Install the client CA chain and test

1. Select Install CA and paste one or more PEM CA certificates used to trust your clients.
2. Keep the HTTP management interface available and schedule a device restart before testing. Installation updates storage but does not actively restart the running service on port 443.
3. Access the relevant test endpoints using a trusted client certificate with the correct purpose and role.
4. Repeat with an untrusted certificate or no certificate, and confirm that the connection is rejected.

Check the result: the device presents the intended new certificate; trusted clients can access only the endpoints allowed for their roles, and untrusted clients cannot connect. Handshake and role behavior must be tested on the device, not inferred from an installation message.

5.5 Renew certificates or delete all credentials

For certificate expiry alone: you can reuse an uncompromised private key to request a new certificate, install it, and test again. However, Config Packs are tied to the recipient certificate's fingerprint. Even with the same private key, a changed certificate fingerprint causes old packages to be rejected as belonging to another recipient after reinitialization or restart. Arrange replacement packages before changing the certificate.

When replacing the private key: the old CSR and certificate no longer match the new key. This does not revoke the old certificate at the CA. If the old key was exposed, handle revocation and incident response separately. Packages that depend on a lost private key may be unrecoverable.

To delete all PKI credentials: Delete Credentials in the certificate section clears the device private key, device certificate, and client CA chain together. Verify your HTTP recovery access and arrange replacement packages before confirming. A backup of the public certificate cannot restore a private key.

After deletion, refresh the page and check for an uninitialized state. Restart and verify that port 443 no longer uses the old credentials. To restore the service, generate a new key, obtain and install a device certificate, install the CA chain, and repeat the tests.

6. Understand Config Pack limitations

A Config Pack is an encrypted, signed `.tscfg` package. The current implementation can create and inspect packages, but general configuration application is unfinished. **Do not rely on it for production fleet configuration, disaster recovery, or proof that settings have changed.**

6.1 What the controls currently do

Action	Current result
Export Device Certificate	Shows the public certificate so a sender can create a package for this device; does not export the private key
Verify	Checks structure and the ciphertext signature against the included certificate; does not establish signer trust or recipient identity
Import after selecting or pasting a package	Frontend and backend parameters do not match; this flow cannot complete
Import from the package list	Validates an existing device file; does not copy, decrypt, or apply it
Apply	Decrypts and lists module names without writing module settings; may still report success
Export Config Pack on a Developer device	Creates a downloadable encrypted, signed package and attempts to save it to the SD card

6.2 Verify the source, not just the signature

The recipient currently checks the signature using the certificate included in the package. Signer certificate-chain trust validation is not implemented. The signature covers the ciphertext; do not assume that every displayed metadata field is therefore authenticated. An Official label is not proof of a trusted source either.

Before importing or inspecting a package, use an asset system or an independent, approved channel to confirm the signer certificate fingerprint, target certificate, and intended change.

Receiving a certificate and its fingerprint together in one email is not an independent check. The target name shown in the preview is not a substitute for checking the certificate fingerprint.

6.3 Share the device certificate and inspect a package

To provide this device's certificate: select Export Device Certificate, copy the full PEM and displayed fingerprint, and send the public certificate to the sender. Confirm the fingerprint through a separate trusted channel.

To inspect a received package: open Import Config Pack, select or paste the `.tscfg` file, and select Verify. Review the signer details and check the source. Verification does not apply settings or prove that this device is the recipient. Stop if the source, target, or purpose is unclear.

Even if verification passes, do not rely on the current general Import and Apply controls to configure the device. Use the supported controls on the relevant feature pages and check the actual settings afterward.

6.4 Export a package from a Developer device

1. Prepare valid JSON configuration files on the SD card and obtain a verified target-device certificate.
2. Select Export Config Pack, choose the files, enter a name and description, and paste the target certificate.
3. Generate and download the `.tscfg` file.
4. Check the browser download and the saved file under `/sdcard/output_config/` separately. If the SD write fails, the API may still return the package for download.

Export does not modify the source settings. Regenerate a package made for the wrong target or an obsolete recipient certificate. Successful export does not establish that the full distribution workflow works: general application on the receiving device is still unfinished.

7. Troubleshooting and incident response

7.1 Common problems

Symptom	What to do
Account Security is missing	Sign in as root; admin does not see this section
Key creation reports success, but the key is missing	Refresh the list and check capacity; do not reuse IDs or repeatedly generate keys
ECDSA deployment fails	Create an RSA key with a new ID; do not repeatedly relax server policy
Deployment succeeds, but Test fails	Check remote authorization through another connection before deploying again
Access still works after removing a host	Remove affects only the local record; revoke the remote public key separately
A fingerprint changes or the server's identity is unclear	Stop connecting and verify through a trusted console; see Section 4.3
Port 443 fails after certificate installation	Check the device key, certificate, and client CA chain; restart and test with a suitable client certificate
A client still rejects the certificate	Check the client trust store, SAN, EKU, validity, and chain; the device's CA store is not the browser's trust store
Apply succeeds, but settings do not change	Module settings are not written by the current implementation; use the relevant feature pages

7.2 Suspected SSH private-key exposure

1. Restrict device and log access. Current logs may contain the beginning of a private key.
2. Revoke the public key on every affected server through trusted connections. Check backup copies and confirm that the old key no longer works.
3. Create and deploy a new RSA key under a new ID. Once it works, delete the old key and exported copies.
4. Review sign-in logs. The device's local list may not include every server where the key was authorized.

If the HTTPS private key was exposed instead, replace the device credentials as described in Section 5.5 and work with the CA administrator on certificate revocation and old packages.

7.3 Other current security risks

In addition to the HTTP, authorization, first-connection trust, and package limitations already described, the inspected build configuration does not enable NVS Encryption, Flash Encryption, or Secure Boot. Do not claim that these mechanisms protect stored private keys or startup integrity. The firmware actually flashed to a device and its eFuse settings require separate checks.

If your use case cannot tolerate these limitations, keep the device out of that network or trust environment until the security owner approves isolation measures or a product fix. Checks in a user guide cannot replace missing product security controls.

8. Review scope and glossary

8.1 Basis of this guide

The review used the current working copy based on `d6ed947`. Page behavior and messages are in `components/ts_webui/web/js/app.js`; API behavior is in `components/ts_api/src/`; key and host storage are in `components/ts_security/src/`. Certificates, the port 443 service, and packages are implemented in `components/ts_cert/`, `components/ts_https/`, and `components/ts_config_pack/`.

Specific checks covered duplicate key IDs, SSH test exit-status handling, local cleanup after revocation, recipient certificate fingerprints, and unfinished package import and application logic. This was a static review: this revision did not connect to remote servers, change device credentials, or perform security acceptance tests on hardware.

8.2 Terms used in this guide

- **Public key / private key:** share the public key with an administrator who grants access; keep the private key secret. Authentication uses the matching pair.
- **NVS:** a Flash storage area for device settings and keys. The name does not imply encryption.
- **CSR / CA:** a certificate signing request, and a certificate authority or its certificate.
- **CN / O / OU:** common name, organization, and organizational unit fields in a certificate subject.
- **SAN / EKU:** the names or IP addresses a certificate covers, and the authentication purposes it permits.
- **PEM:** a text format with BEGIN/END markers, used for certificates, CSRs, and keys.
- **PKI / mTLS:** the certificate and trust-management system, and mutual TLS, in which both the client and server present certificates.